



KTH Technology and Health

TENTAMEN

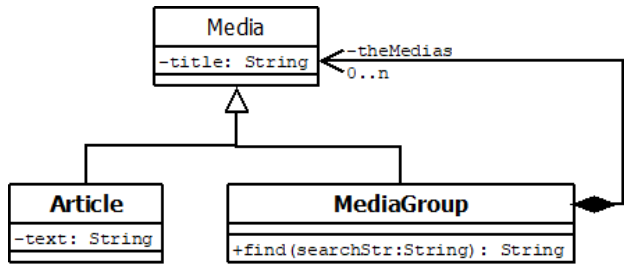
Kurs, kursnummer	HI1027, Objektorienterad programmering
Moment:	TEN1, 3,5 hp
Program: Åk:	TIDAA, TIELA, TIMEL -
Examinator:	Anders Lindström
Rättande lärare:	Anders Lindström
Datum: Tid:	Måndag 2022-10-24 08:00-13:00
Hjälpmedel:	IntelliJ och Netbeans finns på tentamenskontot API-dokumentation och en kort syntaxlista finns i mappen P/Java/api/index.html
Omfattning och betygsgränser:	För betyget E krävs minst hälften av poängen på A-delen samt minst hälften av poängen på B-delen och att minst en av de två första programmeringsuppgifterna är helt korrekt lösta. För betyg C krävs totalt minst 70 % av totala poängen; för betyg A krävs minst 90 % av totala poängen.
Övrig information:	Del A, Teoriuppgifter: Svara på separat papper. Del B, Programmeringsuppgifter: Lösningar lämnas på tentamenskontot. <i>När du skapar projektet i IntelliJ eller NetBeans måste du kontrollera att projektet hamnar under VPROVXXXX/H, ditt tentamenskonto. Alla lösningar skrivs i ett och samma projekt, men varje uppgift ska ligga i ett separat paket, med namnen uppg 9, uppg10, ...</i> Innan du avslutar tentamen: Kontrollera att du verkligen sparar dina lösningar under VPROVXXXX/H

För att komma åt måsvingar, hakparenteser och "|", använd "AltGr" + lämplig tangent,
t.ex. "AltGr" + "{" (7:ans tangent).

Del A, Teoriuppgifter

Svara, på separat papper, kort men koncist på frågorna. Motivera alltid dina svar.

Otydliga lösningar, både vad gäller läsbarhet och begriplighet, rättas inte.

- Vilken synlighet får man om varken private, protected eller public anges?
 - Vad skiljer denna synlighetsnivå från protected?(2 p)
- Vad menas, inom objektorienterad programmering, med begreppet hög kohesion? Vad är poängen med att skriva klasser med hög kohesion?(2 p)
- Antag att a och b är referensvariabler. Vad är skillnaden mellan att jämföra med "a == b" respektive med "a.equals(b)"?(2 p)
- Antag att vi vill skriva en klass som representerar en kortlek, Deck. Klassen ska naturligtvis garantera att det finns 52 unika kort när kortleken fyllts och att det endast går att ta det kort som ligger överst i leken. Vid implementationen av Deck kan man använda klassen ArrayList. Vad blir problemet om vi implementerar Deck-klassen som en subklass till ArrayList?(2 p)
- Förklara begreppet polymorfism. Vad är förutsättningen för att vi ska kunna ha polymorf kod?(2 p)
- Klassdiagrammet beskriver ett elektroniskt nyhetssystem där det finns nyhetsgrupper och artiklar. Förklara vad relationssymbolerna i figuren innebär i praktiken vad gäller relationer mellan objekten och klasserna. Vilket designmönster är detta ett exempel på?

```
classDiagram
    class Media {
        -title: String
    }
    class Article {
        -text: String
    }
    class MediaGroup {
        +find(searchStr:String): String
    }
    Media <|-- Article
    Media <|-- MediaGroup
    MediaGroup "0..n" *-- "1" Media : -theMedias
```

(2 p)
- Förklara hur nyckelorden "throw" respektive "throws" används i språk som Java.
 - Vad är poängen med att ha olika (sub-) typer av exceptions?(2 p)
- Ange en situation där metoderna wait och notify/notifyAll, från klassen Object, kan användas på ett meningsfullt sätt. Det måste framgå var, och varför, anropen sker samt vad de innebär för det/de inblandade kodflödet/kodflödena.(2 p)

Del B, Programmeringsuppgifter

Lösningar lämnas i ett gemensamt projekt på tentamenskontot. För varje lösning ska det finnas ett paket med namnet "uppg9", "uppg10" etc. Kontrollera att du verkligen sparar dina lösningar på H.

Till dessa uppgifter kan du i vissa fall ha hjälp av Javas API-dokumentation, som finns på P:/Java/api. *Alla klasser ska följa objektorienterade principer, speciellt inkapsling. Om inte annat anges ska alla klasser ha en konstruktör som initierar alla datamedlemmar, access-metoder samt en omdefinierad version av toString.*

9. Fordon

Du ska skriva kod för att hantera fordon (vehicles) och ägare till dessa. En person kan äga från inget till många fordon.

Klassen Vehicle representerar ett fordon. Ett fordon har en typ, representerad av en enum (med värden car, truck och motorbike), en modell (text) samt ett registreringsnummer (av typ RegNumber, se nedan). Objekt av typen Vehicle är immutable.

Registreringsnumret ska representeras av en klass RegNumber, vars objekt ska vara immutable. Objekt skapas via en klass-metod, getInstance, som tar en text som argument, och först kontrollerar om indata är korrekt (tre stora bokstäver A-Z följt av tre siffror)¹. Om indata inte är korrekt ska ett IllegalArgumentException kastas, i annat fall returneras ett nytt objekt. Konstruktorn ska vara privat.

Klassen Person representerar en ägare. En ägare har ett namn och en adress (text) – adressen kan ändras, men inte namnet. En ägare har dessutom en lista med ägda fordon och en tillhörande accessmetod som returnerar en kopia av listan.

Fordon läggs till en ägare med addVehicle(Vehicle v). Om fordonet redan ägs av denna person ska ett IllegalStateException kastas. Det ska även finnas en metod boolean removeVehicle(Vehicle v) som tar bort ett fordon ur listan.

Skriv även en main-metod där du demonstrerar funktionaliteten.

(4 p)

10. Former

Skriv den abstrakta klassen Shape som representerar det som är gemensamt för ritbara figurer. Shape ska ha *privata* datamedlemmar för position, se nedan, samt för färg, som representeras av en enum med värden red, green och blue. Det ska vara möjligt både att avläsa och att ändra värdet på datamedlemmarna. Klassen ska även ha en abstrakt metod, paint, för att rita objekt.

Position representeras med klassen Point, med datamedlemmarna x och y (heltal). Objekt av typen Point ska vara immutable, oföränderliga. Klassen ska även implementera interfacet java.lang.Comparable, så att rangordningen bestäms av x-värdet men om dessa är lika av y-värdet, samt omdefiniera Object.equals.

Metoden toString ska ge en textsträng på formen "[10, 20]", x- och y-värdet.

¹ Du kan t.ex. använda dig av regex: String regStrPattern = "[A-Z]{3}[1-9]{3}"; boolean matches = regStr.matches(regStrPattern);

Skriv sedan två subklasser till Shape, enligt nedan. Båda klasserna ska ha meningsfulla konstruktörer, metoder för att avläsa och ändra data samt en implementation av paint.

- Line har förutom ärvt data ytterligare en punkt, linjens ändpunkt. Metoden paint ska implementeras så att den ger en utskrift på formen "Line: [10,10], [20,30], GREEN".
- Circle har förutom ärvt data en diameter, ett heltal. Metoden paint ska implementeras så att den ger en utskrift på formen "Circle: [30,5], 20, BLUE".
Diametern får inte vara negativ – alla metoder som sätter/ändrar diametern ska kasta ett IllegalArgumentException om indata är ett negativt värde.

Skriv sedan en main-metod där du skapar en lista med Shape-referenser, lägger in Line- och Circle-objekt samt demonstrerar funktionaliteten för dessa.

(4 p)

11. Nyligen

Interfacet IRecentList, se sista sidan, definierar funktionalitet för en speciell sorts listklass som har nedanstående egenskaper.

- Listan är generisk.
- Listan innehåller inga dubletter. Metoden equals används för att avgöra om två objekt är lika.
- Metoden add lägger till ett objekt först i listan alternativt, om objektet redan finns i listan, flyttar det till första positionen.
- Metoden get returnerar en referens till objektet på angivet index. Objektet ska inte tas bort från listan, men flyttas till första positionen.

Övriga metoder, och deras kontrakt, framgår av kommentarer i interfacet, se sista sidan.

Skriv en klass, RecentList, som implementerar detta interface. I din implementation ska du använda dig av en ArrayList eller en LinkedList. Klassen ska även ha en toString-metod som ger en text på formen "The objects: [..., ..., ..., ...]" samt en konstruktor. Ingen annan (publik) funktionalitet får finnas.

Skriv sedan en main-metod där du instansierar listan i två olika fall, med typparametern satt till String respektive till Integer, och demonstrerar funktionaliteten.

I fallet String ska man, om man i tur och ordning lägger till "A", "B", "C", "B", "D", "E" och "A", få följande innehåll i listan: [A], [B,A], [C,B,A], [B,C,A], [D,B,C,A], [E,D,B,C,A] och [A,E,D,B,C].

(4 p)

12. Funktionellt

Du ska skriva funktionalitet för att utifrån en lista med objekt beräkna ett värde, dvs. reducera en samling värden till ett enda. Exempelvis ska vi kunna anropa metoden med en lista av heltal (representerade via wrapper-klassen Integer) och som resultat få summan av talen.

För detta behöver du skriva ett interface, Reducer, med en metod, apply, som tar två objekt som argument, det hittills beräknade värdet (det ackumulerade) och ett nytt värde, av samma typ, och sedan returnerar ett nytt värde beräknat utifrån dessa två argument. Interfacet ska vara generiskt.

Metoden för att konvertera en hel lista till ett enda värde, reduce, skrivs förslagsvis som en klassmetod, i en separat klass, med nedanstående signatur.

```
public static <T> T reduce(List<T> values, Reducer<T> reducer, T init)
```

"init" representerar utgångsvärdet, i fallet summering av tal ska init sättas till 0.

Skapa sedan de tre implementationer av interfacet som beskrivs nedan.

1. Sum, som tar ett tidigare beräknat heltalsvärde lägger till ett nytt värde och returnerar summan. Använd klassen Integer för att representera heltalen.
2. SumWithinLimits, som tar ett tidigare beräknat heltalsvärde och lägger till det nya värdet endast om det ligger inom ett givet intervall. Implementationen behöver en konstruktor där intervallet definieras.
3. Concat, som tar en tidigare beräknad textsträng och lägger till en ny textsträng, med ett mellanslag, och returnerar den nya textsträngen.

Skriv en main-metod som demonstrerar funktionaliteten. Metoden reduce ovan ska returnera en summa av talen i listan om första eller andra implementation används. Om tredje implementationen används ska en text sammanslagen av deltexter från listan returneras.

(4 p)

```

/**
 * A list of objects, no duplicates (according to equals) are allowed.
 * The most recent used (via get) object is always at the
 * first position.
 * @param <T> the type of objects in the list.
 */
public interface IRecentList<T> {

    /**
     * Return the number of elements in this list.
     */
    int size();

    /**
     * Return whether this list is empty or not.
     */
    boolean isEmpty();

    /**
     * Return a copy of the internal list of objects.
     * @return a shallow copy of the list.
     */
    List<T> getObjects();

    /**
     * Add an object first in this list or, if the object, according to
     * equals, already
     * exists in this list, move the object to the first position in this
     * list.
     * @param obj the object to add
     */
    void add(T obj);

    /**
     * Return the object at the indicated position, without removing it
     * from this list.
     * Move the returned object to the first position in this list.
     * @param index the position of the object
     * @return the object at position index
     * @throws IndexOutOfBoundsException if index is out of bounds. Add a
     * message indicating the
     * erroneous index.
     */
    T get(int index);

    /**
     * Remove the indicated object from this list - use equals to check for
     * equality.
     * @param obj the object to remove
     * @return true if the object was removed.
     */
    boolean remove(T obj);
}

```